Chandubhai S Patel Institute of Technology

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGYAND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement:

- Integrate a public RESTAPI (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.
- Use the Projects page built in Practical 2 as the integration point.
- Implement a loading spinner component shown while the request is in progress.
- Implement an error message component shown if the API call fails.
- Render at least the repository name and URL for each item returned.

Screenshots:

Practical 3 - API Integration | ErrorMessage.jsx

src > components > ErrorMessage.jsx > ...

```jsx
function ErrorMessage({ message, onRetry }) {
  return (
    <div className="status error-state">
      <p>{message}</p>
      <button onClick={onRetry} type="button">
        Retry
      </button>
    </div>
  );
}

export default ErrorMessage;
```

Figure 1ErrorMessage.jsx Error message component displaying the error text along with a "Retry" button that triggers the onRetry callback.

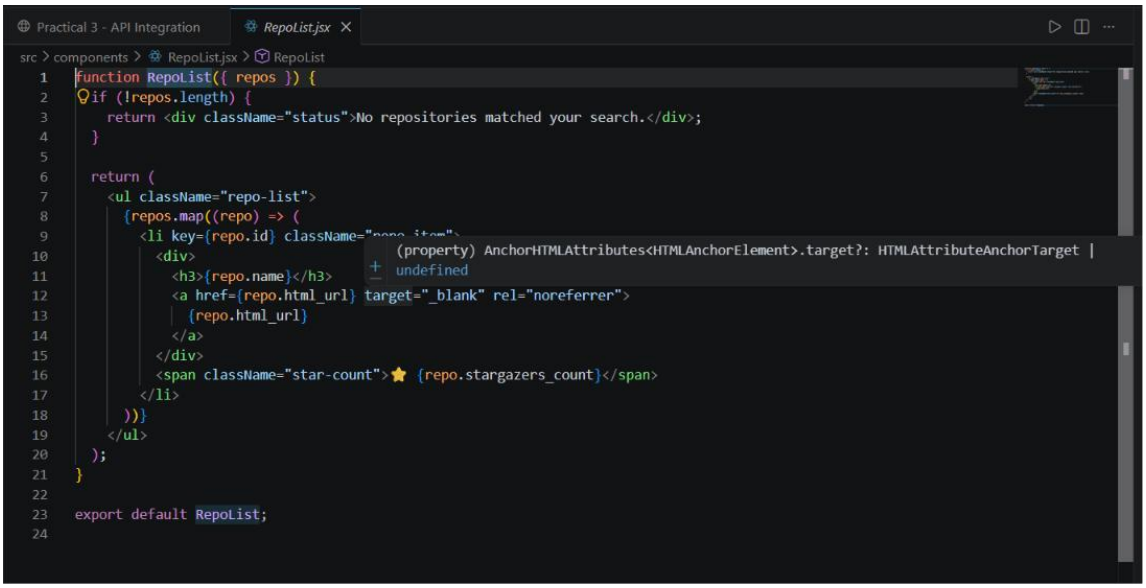


Figure 2RepoList.jsx Repository list component that renders each repo's name, clickable URL, and star count, with a fallback message when no repositories match the search.



Figure 3Loading spinner component shown while the API request is in progress ("Loading repositories...").

Practical 3 - API Integration — App.jsx

src > App.jsx > App > filteredRepos > useMemo() callback

```jsx
import { useEffect, useMemo, useState } from 'react';
import Spinner from './components/Spinner';
import ErrorMessage from './components/ErrorMessage';
import RepoList from './components/RepoList';

const DEFAULT_USERNAME = 'octocat';

function App() {
  const [repos, setRepos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [searchTerm, setSearchTerm] = useState('');
  const [username, setUsername] = useState(DEFAULT_USERNAME);

  const fetchRepos = async (selectedUsername) => {
    setLoading(true);
    setError('');

    try {
      const response = await fetch(
        `https://api.github.com/users/${selectedUsername}/repos?per_page=100`
      );

      if (!response.ok) {
        throw new Error(`Unable to fetch repositories for ${selectedUsername}.`);
      }

      const data = await response.json();
      setRepos(data);
    } catch (err) {
      setRepos([]);
      setError(err.message || 'Something went wrong while fetching repositories.');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchRepos(DEFAULT_USERNAME);
```

Practical 3 - API Integration — App.jsx

src > App.jsx > App > filteredRepos > useMemo() callback

```jsx
function App() {
  const fetchRepos = async (selectedUsername) => {
      setError(err.message || 'Something went wrong while fetching repositories.');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchRepos(DEFAULT_USERNAME);
  }, []);

  const filteredRepos = useMemo(() => {
    return repos.filter((repo) =>
      repo.name.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }, [repos, searchTerm]);

  const handleRetry = () => {
    fetchRepos(username);
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    fetchRepos(username);
  };

  return (
    <main className="app-shell">
      <section className="card">
        <header className="hero">
          <div>
            <p className="eyebrow">Practical 3</p>
            <h1>API Integration and Data Rendering in React</h1>
            <p className="subtitle">
              This page demonstrates loading, error, retry, search, and repository list rendering with the GitHub RES
            </p>
          </div>
        </header>
```

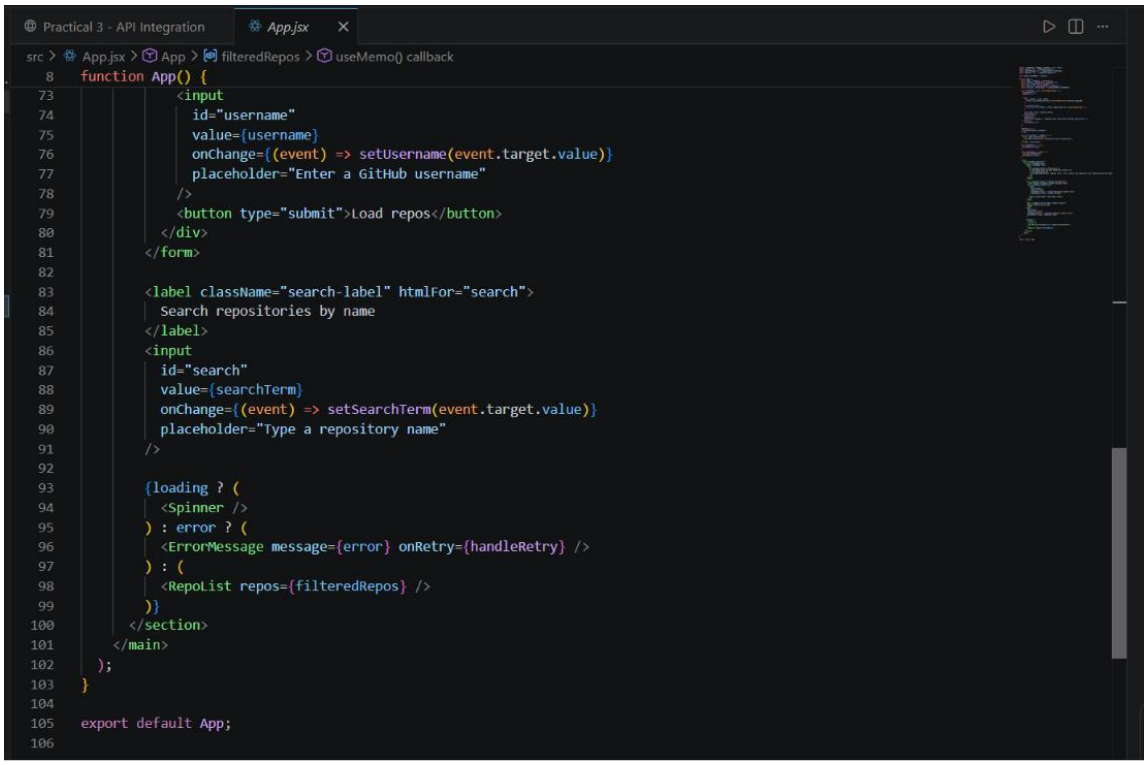


Figure 4fetchRepos async function using fetch() to call the GitHub API, with try/catch/finally handling for success, error, and loading state updates.

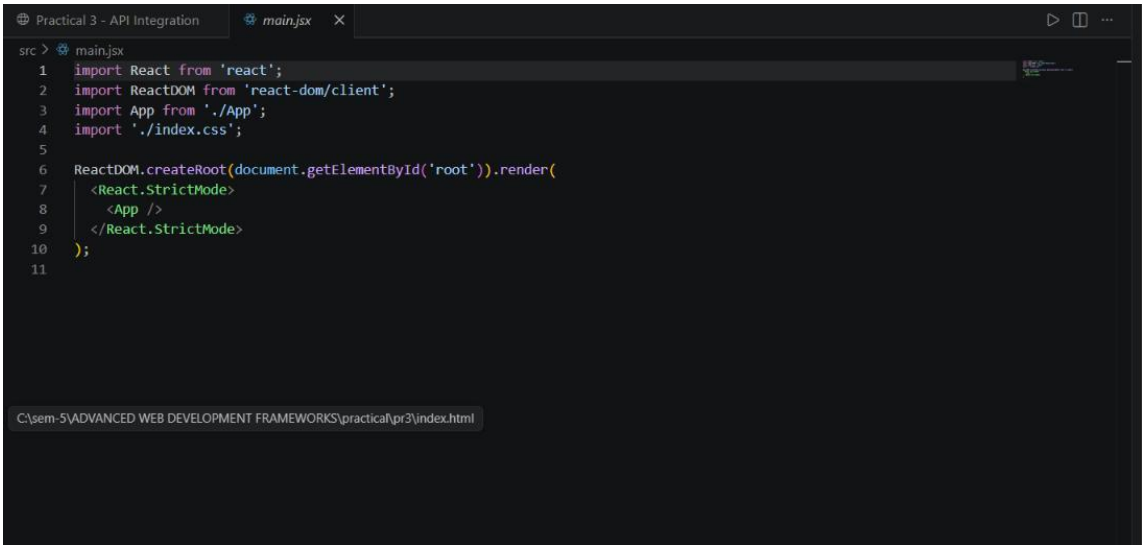


Figure 5React app entry point rendering the <App /> component into the DOM inside React.StrictMode.

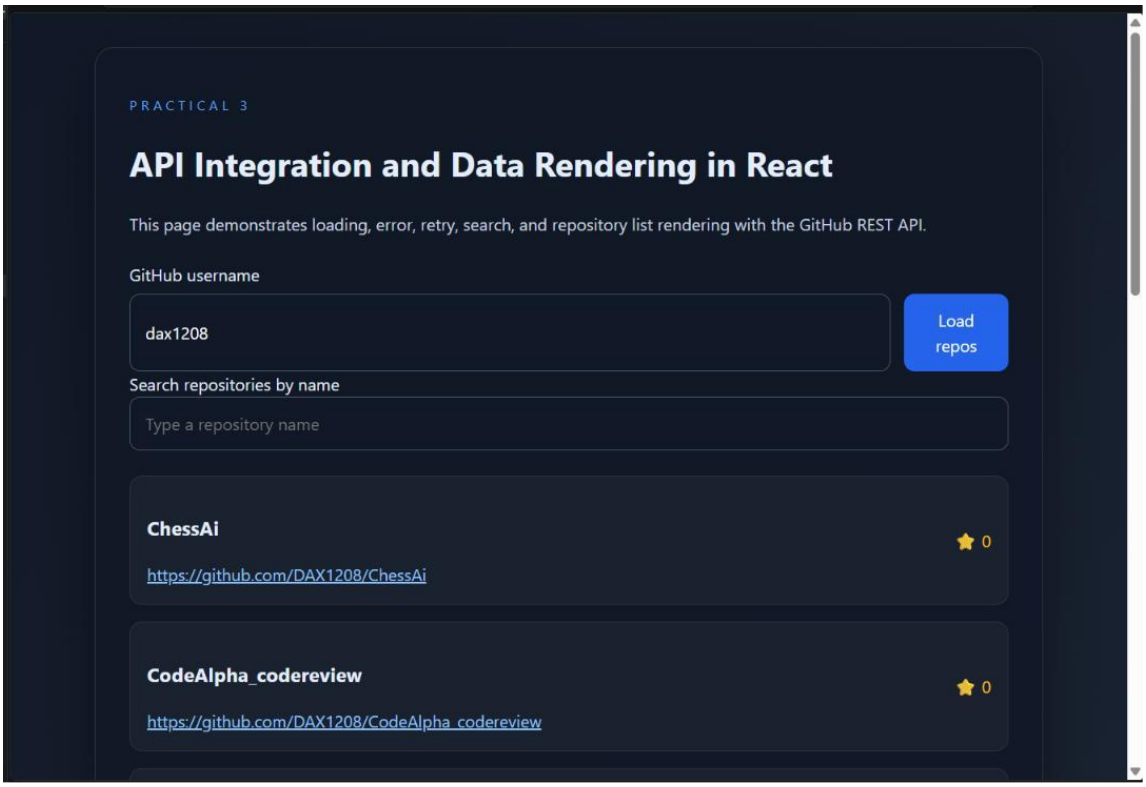


Figure 6Final rendered UI showing the GitHub username input, search bar, and a dynamically loaded list of repositories with names, links, and star counts.

Github repo:

portfolio.githttps://github.com/sahilmak1602/WDF_24IT045_SAHIL.git